

Twilio Voice API Integration

Smart Event Crowd Management — SOS emergency contact calling

Goal: When an attendee presses SOS, notify the event admin and request an automated voice call to the attendee's registered trusted contact.

This guide combines account setup, credentials, backend implementation, frontend testing, and the SOS workflow into one document.

Prototype guide • Node.js + Express backend • React-compatible frontend

1. Before you begin

You will need a Twilio account, a verified test phone number, Node.js installed, and a project folder for your backend. The examples below assume your frontend can send HTTP requests to a Node.js/Express server.

Important trial limitation: Twilio trial accounts generally restrict outbound calls to verified recipient numbers. Trial features and destination-country restrictions can vary. Test with a number you control and verify the current rules in your Twilio Console before using the API.

Useful official pages

Twilio Console: <https://console.twilio.com/>

Twilio Voice API overview: <https://www.twilio.com/docs/voice/api>

Twilio Node.js quickstart: <https://www.twilio.com/docs/voice/quickstart/node>

Trial account restrictions: <https://www.twilio.com/docs/usage/tutorials/how-to-use-your-free-trial-account>

2. Get your Twilio credentials

1. Sign in to the Twilio Console.
2. Open the account dashboard and locate your Account SID and Auth Token, or create an API key and secret for backend use.
3. Locate your Twilio phone number, if one is available for voice calling in your account and destination country.
4. In the Console, open the Voice trial/demo area and make a test call to a verified number.

Keep all credentials on the server. Never place the Auth Token or API secret in React code, a public repository, or a mobile app bundle.

3. Create the backend project

Open a terminal in a new backend folder and run:

```
npm init -y
npm install express twilio dotenv
```

Create a file named **.env** in that folder:

```
TWILIO_ACCOUNT_SID=your_account_sid
TWILIO_AUTH_TOKEN=your_auth_token
TWILIO_PHONE_NUMBER=your_twilio_voice_number
```

Replace the placeholders with values from your Twilio Console. Add **.env** to **.gitignore** so it is not uploaded to source control.

```
.env
node_modules/
```

4. Create the voice-call API

Create **server.js**. This demonstration endpoint accepts a phone number and asks Twilio to place a text-to-speech call. It is a prototype only: protect it with authentication and authorization before exposing it to users.

```
require("dotenv").config();

const express = require("express");
const twilio = require("twilio");

const app = express();
app.use(express.json());

const client = twilio(
  process.env.TWILIO_ACCOUNT_SID,
  process.env.TWILIO_AUTH_TOKEN
);
```

```

app.post("/api/test-call", async (req, res) => {
  try {
    const { phoneNumber } = req.body;

    if (
      typeof phoneNumber !== "string" ||
      !/^\\+[1-9]\\d{7,14}$/.test(phoneNumber)
    ) {
      return res.status(400).json({
        error: "Enter a valid number in international format."
      });
    }

    const call = await client.calls.create({
      to: phoneNumber,
      from: process.env.TWILIO_PHONE_NUMBER,
      twiml: `<Response>
        <Say>
          This is a test alert from the Smart Event Crowd
          Management system. Please check with the attendee.
        </Say>
      </Response>`
    });

    res.json({
      message: "Call request submitted",
      callSid: call.sid
    });
  } catch (error) {
    console.error("Twilio error:", error.message);
    res.status(500).json({
      error: "Could not start the call. Check Twilio settings."
    });
  }
});

app.listen(3000, () => {
  console.log("Backend running on port 3000");
});

```

Run the server:

```
node server.js
```

Note: Twilio requires a valid caller ID/number authorized for your account. Some trial accounts or countries may not support the exact call setup shown above. If this request fails, inspect the error in your backend terminal and follow the Voice API/Console guidance for your account.

5. Call the backend from your frontend

For a React frontend, you can test the endpoint with this function. Replace the example phone number with a number you are authorized to call and that is verified for your trial.

```
async function testCall() {
  const response = await fetch("/api/test-call", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      phoneNumber: "+91XXXXXXXXXX"
    })
  });

  const result = await response.json();

  if (!response.ok) {
    throw new Error(result.error || "Call request failed");
  }

  console.log(result);
}
```

Connect **testCall()** to a clearly labelled test button. If your frontend and backend run on different origins during development, configure the correct backend URL and restrict CORS to your own frontend origin.

6. Connect the call to the SOS workflow

For the real Smart Event Crowd Management flow, do not let the browser submit any arbitrary recipient number. The server should retrieve the trusted contact saved for the authenticated attendee.

1	Attendee presses SOS in the website/app.
2	Backend authenticates the attendee and validates the request.
3	Backend creates an incident record with timestamp, event, attendee ID, and status.
4	Backend notifies the event admin/dashboard.
5	Backend retrieves the attendee's registered trusted contact and requests a Twilio call.
6	Backend records the returned Call SID and checks call status through callbacks/webhooks.
7	Admin coordinates the appropriate event response; the trusted-contact call is supplementary.

7. Safety, privacy, and reliability checklist

- Obtain clear consent from attendees and trusted contacts before sending automated alerts.
- Verify phone numbers and limit repeated calls to prevent accidental spam or abuse.
- Protect SOS and test-call endpoints with authentication, authorization, rate limiting, and server-side validation.
- Store only the data you need; restrict admin access and protect incident records.
- Track call outcomes—request accepted does not mean the person answered.
- Provide a separate, clearly visible way to contact local emergency services. In India, 112 is the pan-India emergency number; do not make the automated trusted-contact call a substitute for emergency services.
- For a school/college demonstration, simulate the call until provider permissions and country-specific restrictions are confirmed.

8. Common troubleshooting

Call rejected on trial: verify the destination number in the Twilio Console and check country/destination restrictions.

Invalid from number: confirm the number is a Twilio voice-capable number available to your account and

suitable for the destination.

Authentication error: re-check Account SID and Auth Token/API credentials; do not print secrets in logs.

Frontend request fails: confirm the backend is running, the URL is correct, and CORS is configured if the origins differ.

Call initiated but not answered: use call-status callbacks/webhooks and show an unanswered status to the admin.

9. Official documentation

Voice API: [Twilio Voice API](#)

Node.js quickstart: [Twilio Node.js quickstart](#)

Trial account: [Twilio free trial account guidance](#)

India emergency response: [112 India](#)

This guide is a development starting point, not a production-ready emergency dispatch system. Test carefully and confirm the current Twilio trial rules, local telecom requirements, and provider support before using real contact numbers.